08

Filas e processamento em segundo plano

Redis, workers e processamento assíncrono no Laravel

Como tirar tarefas lentas da requisição principal sem perder rastreabilidade.

Roteiro da aula

- Continuidade do projeto de cache
- Job, fila e worker
- Criação de um job
- Encadeamento de jobs
- Processamento em lote
- Middleware de jobs
- Falhas, tentativas e backoff
- Laravel Horizon
- Projeto do módulo

Fluxo assíncrono da API

Contexto: Na aula anterior, o Redis acelerou leituras com cache.
Agora: A API grava um job no Redis e responde antes do processamento.
Exemplo: POST /api/reports retorna 202 Accepted com job_id para consulta.

```
POST /api/reports
202 Accepted
{
  "job_id": "01J...",
  "status": "queued",
  "queue": "reports"
}
```

Job, fila e worker

Job: Descreve os dados e a tarefa.
Fila: Mantém o job até um worker estar disponível.
Worker: Reserva, executa e remove o job concluído.
Exemplo: GenerateTaskReport entra na fila reports.

```
QUEUE_CONNECTION=redis

GenerateTaskReport::dispatch($userId)
    ->onQueue('reports');

php artisan queue:work redis --queue=reports
```

Criação de um job

Responsabilidade: O job recebe apenas dados serializáveis e delega a regra ao serviço.

Execução: O container injeta ReportService no método handle().

Exemplo: O worker gera o relatório para userId.

Exemplo de código

```
final class GenerateTaskReport implements ShouldQueue
{
    use Queueable;

    public function __construct(public int $userId) {}

    public function handle(ReportService $reports): void
    {
        $reports->generate($this->userId);
    }
}
```

Encadeamento de jobs

Objetivo: Executar etapas dependentes na ordem.
Comportamento: O Laravel só inicia a próxima etapa quando a anterior termina.
Falha: Se GenerateTaskReport falhar, NotifyReportReady não executa.

Exemplo de código

```
Bus::chain([
    new GenerateTaskReport($userId),
    new NotifyReportReady($userId),
])->onQueue('reports')->dispatch();
```

Processamento em lote

Objetivo: Acompanhar vários jobs independentes como uma operação.
Estado: O batch_id permite consultar total, pendentes, falhas e progresso.
Exemplo: Criar um ReindexTask para cada tarefa aberta.

Exemplo de código

```
$jobs = Task::where('status', 'open')
    ->pluck('id')
    ->map(fn (int $id) => new ReindexTask($id));

$batch = Bus::batch($jobs)
    ->onQueue('reports')
    ->dispatch();

return response()->json([
    'batch_id' => $batch->id,
    'status' => 'queued',
], 202);
```

Middleware de jobs

Objetivo: Aplicar regras de execução sem misturá-las ao handle().

Redis: RateLimitedWithRedis compartilha o contador entre workers.

Exemplo: O job excedente volta à fila e tenta novamente.

Exemplo de código

```
public function middleware(): array
{
    return [
        (new RateLimitedWithRedis('report-api'))
            ->releaseAfter(10),
    ];
}
```

Falhas, tentativas e backoff

Falha temporária: O worker libera o job para uma nova tentativa.

Backoff: Os intervalos aumentam para não pressionar a dependência.

Falha final: Após três tentativas, failed() registra o erro.

Exemplo de código

```
public int $tries = 3;

public function backoff(): array
{
    return [10, 30, 60];
}

public function failed(Throwable $error): void
{
    Log::error($error->getMessage());
}
```

Laravel Horizon

Painel: Mostra filas, throughput, duração, jobs recentes e falhas.
Operação: Supervisores mantêm workers ativos e equilibram processos.
Exemplo: Acompanhar reports e repetir um job que falhou.

```
php artisan horizon
php artisan horizon:status

Queue: reports
Job: GenerateTaskReport
Status: completed
```

Projeto do módulo

Entrega: Três endpoints tornam o processamento observável.
Criação: Os comandos retornam 202 com um identificador.
Consulta: O cliente acompanha status e progresso sem bloquear a requisição.

```
POST /api/reports
202 { job_id, status }

POST /api/tasks/reindex
202 { batch_id, status }

GET /api/jobs/{id}
200 { status, progress }
```

Checklist final

API: Responde 202 e devolve um identificador.
Redis: Recebe o job na fila correta.
Worker: Conclui handle() ou registra a falha.
Observação: Testes e Horizon mostram o resultado.

```
php artisan test

php artisan queue:work redis --queue=reports --once

php artisan queue:failed
php artisan horizon:status
```